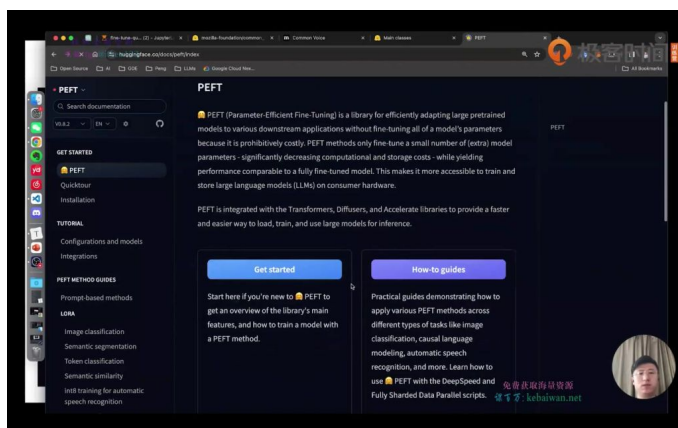


一、PEFT 00:05

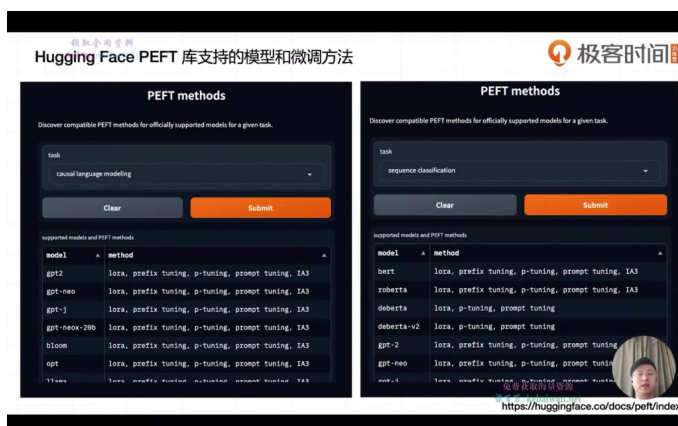
1. PEFT库支持的模型和微调方法 02:04

1) PEFT库介绍



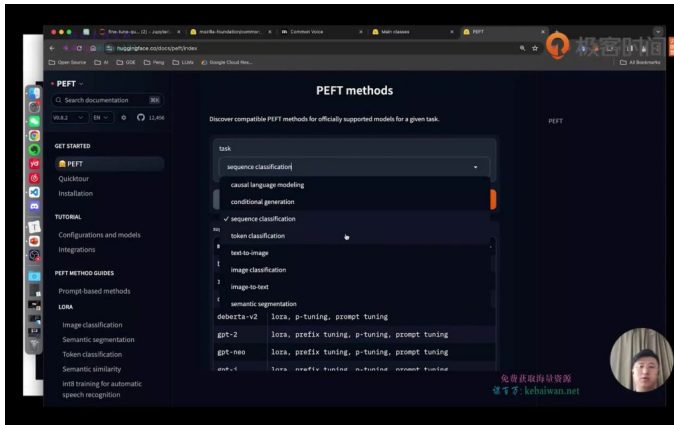
-
- **核心功能：**用于高效适配大型预训练模型到下游任务，无需全量微调所有参数
- **技术优势：**
 - 仅微调少量额外参数，显著降低计算和存储成本
 - 性能接近全量微调模型
 - 支持在消费级硬件上训练和存储大语言模型(LLMs)
- **集成生态：**与Transformers、Diffusers和Accelerate库深度集成，提供更便捷的大模型加载、训练和推理方式

2) PEFT库支持的模型和微调方法：因果语言建模 02:15

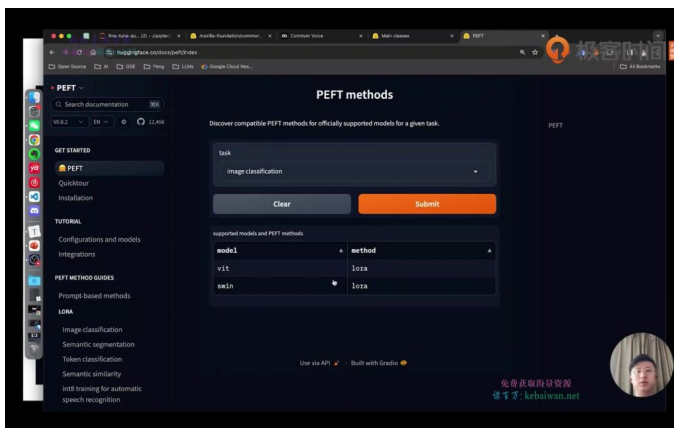


-
- **任务特点：**文本生成类任务（如GPT系列模型应用场景）
- **典型模型支持：**
 - GPT系列：gpt2（支持lora/prefix tuning/p-tuning/prompt tuning/IA3）
 - Bloom/OPT：全量方法支持
 - 新模型示例：mistral（目前仅支持LoRA）
- **方法共性：**所有模型均支持LoRA方法，部分新模型支持方法较少

3) PEFT库支持的模型和微调方法：序列分类 03:48



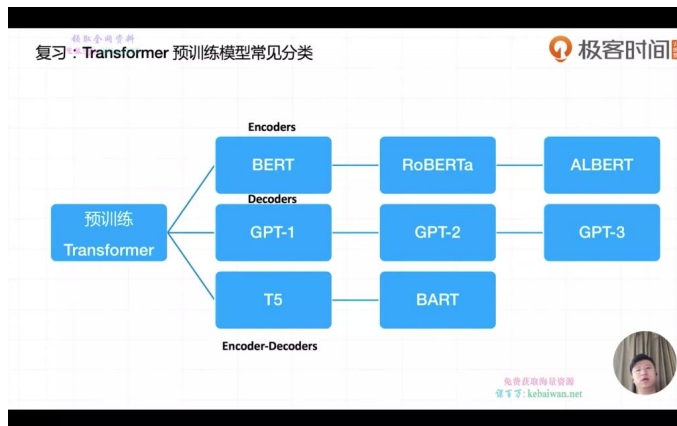
- 典型应用：情感分析等文本分类任务（如BERT模型应用场景）
 - 模型支持特点：
 - BERT/RoBERTa：全方法支持（含IA3）
 - DeBERTa系列：不支持prefix tuning和IA3
 - GPT系列跨界支持：部分生成模型也可用于分类任务
 - 实践案例：课程中曾用BERT进行IMDb评论分类的全量微调，PEFT库可实现相同任务的高效微调
- 4) PEFT库支持的模型和微调方法：图像分类 05:01



- 跨模态支持：
 - ViT (Vision Transformer)：支持LoRA
 - Swin Transformer：支持LoRA
 - 方法通用性：LoRA因其通过低秩矩阵分解（ $W = BA$ 或SVD）降低参数量的通用原理，可跨文本/视觉模态应用
 - 技术特点：使用小型权重矩阵替代高维参数矩阵，保持原模型结构的同时显著减少可训练参数
- 5) PEFT与Transformers的集成问题 05:59

- 协同关系：PEFT专注于高效微调，需与Transformers配合完成完整训练流程
- 集成方式：
 - 共享模型加载接口
 - 统一训练流程管理
 - 联合推理部署
- 实践价值：用户可复用已有Transformers知识体系，仅需额外掌握PEFT的适配层配置

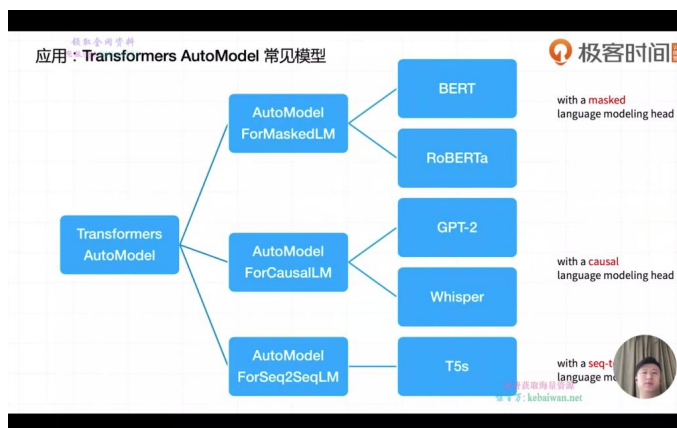
2. Transformer预训练模型常见分类 06:22



-
- **编码器(Encoders):**
 - 代表模型: BERT、RoBERTa、ALBERT
 - 特点: 采用双向注意力机制, 适合理解任务
- **解码器(Decoders):**
 - 代表模型: GPT-1/2/3
 - 特点: 单向注意力, 适合文本生成
- **编码器-解码器(Encoder-Decoders):**
 - 代表模型: Transformer、T5、BART
 - 特点: 兼具理解和生成能力

3. Transformers AutoModel框架 07:27

1) 基类与统一框架 07:35



-
- **AutoModel基类:**
 - 功能: 统一管理所有基于Transformer的模型
 - 优势: 通过相同接口调用不同结构模型
- **Head替换机制:**
 - 原理: 保持主干网络不变, 替换顶部结构
 - 示例: BERT可适配分类/生成等不同任务

2) 精细分类结构 08:05

- **Encoder结构:**
 - 训练特点: 使用掩码语言建模(Masked LM)
 - 代表Head: AutoModelForMaskedLM
- **Decoder结构:**
 - 训练特点: 因果语言建模(Causal LM)
 - 代表Head: AutoModelForCausalLM

- **Encoder-Decoder结构:**
 - **训练特点:** 序列到序列建模(Seq2Seq LM)
 - **代表Head:** AutoModelForSeq2SeqLM

3) BERT模型特性 08:16

- **核心结构:**
 - **注意力机制:** 双向Transformer编码器
 - **预训练任务:** 完形填空式掩码预测
- **典型应用:**
 - 序列分类 (如情感分析)
 - 序列生成 (如文本摘要)

4) GPT模型特性 09:38

- **核心结构:**
 - **注意力机制:** 单向Transformer解码器
 - **预训练任务:** 自回归文本生成
- **典型应用:**
 - 文本续写
 - 对话生成

5) T5模型特性 10:01

- **核心结构:**
 - **创新点:** Text-to-Text统一框架
 - **特点:** 所有任务都转化为文本生成形式
- **典型应用:**
 - 机器翻译
 - 问答系统

6) 模型适配技巧 10:30

- **Head选择原则:**
 - **匹配性:** 根据下游任务选择对应Head
 - **灵活性:** 支持自定义Head开发
- **微调示例:**
 - 使用AutoModelForSequenceClassification做文本分类
 - 使用AutoModelForTokenClassification做NER

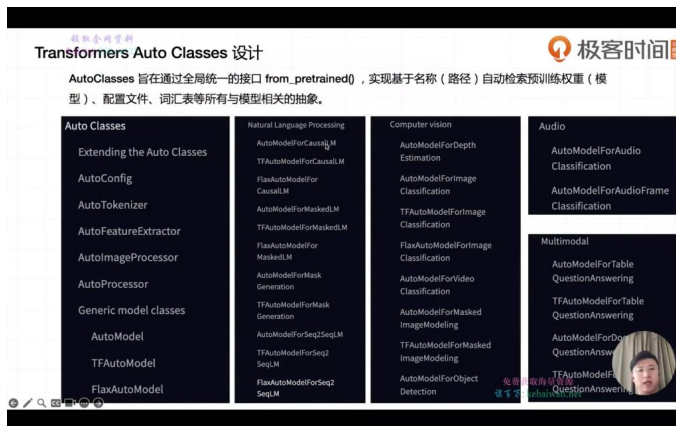
7) 注意力机制本质 12:48

- **核心能力:**
 - **语义理解:** 通过自注意力捕捉上下文关系
 - **结构无关性:** 不同架构均可实现语义理解
- **实践启示:**
 - 模型选择应基于任务需求而非绝对优劣
 - 通过微调可使同模型适应不同任务

4. Transformers Auto Classes设计 13:53

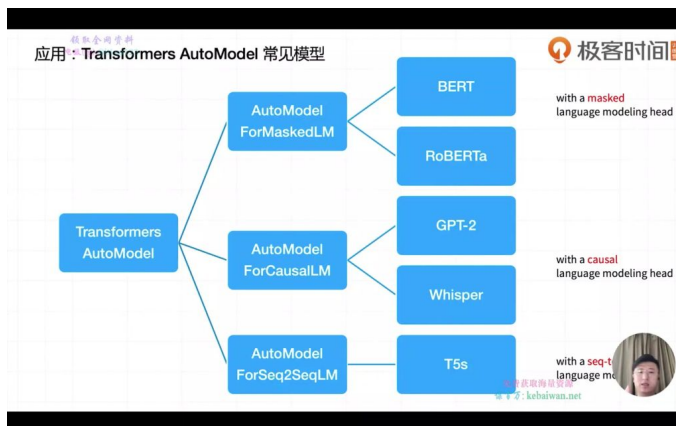
- **核心目标:** 通过全局统一的接口from_pretrained()实现基于名称自动检索预训练权重、配置文件、词汇表等模型相关资源
- **设计特点:** 采用抽象层封装, 支持跨模态 (NLP/CV/Audio/Multimodal) 和跨框架 (PyTorch/TensorFlow/JAX) 的统一调用

1) Auto Classes的分类与基础类 14:00



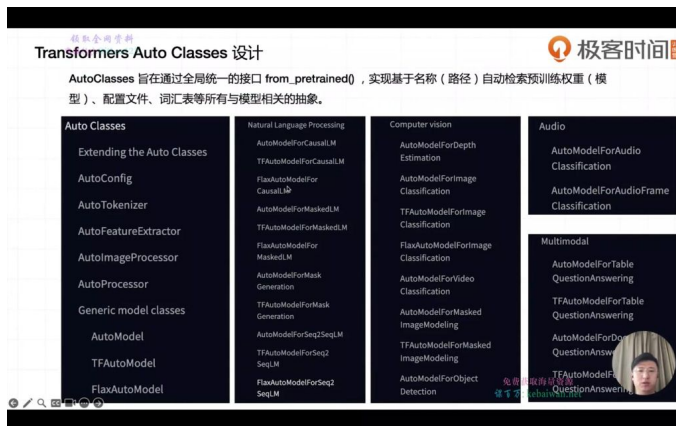
- 模态维度分类：
 - 自然语言处理：包含AutoModelForCausalLM、AutoModelForMaskedLM等
 - 计算机视觉：包含AutoModelForDepthEstimation、AutoModelForImageClassification等
 - 音频处理：包含AutoModelForAudioClassification等
 - 多模态：包含AutoModelForTableQuestionAnswering等
- 基础架构：
 - 核心容器：AutoModel作为基础类，可容纳任何Transformer架构网络
 - 扩展机制：通过替换顶层head实现不同下游任务，如for CausalLM/for MaskedLM等子类

2) Transformers AutoModel常见模型 15:02



- 典型组合：
 - BERT + MLM头 → AutoModelForMaskedLM
 - GPT-2 + CLM头 → AutoModelForCausalLM
 - T5 + Seq2Seq头 → AutoModelForSeq2SeqLM
 - Whisper → 音频处理专用架构
- 实现原理：所有子类最终需落地为具体框架代码，原始实现默认基于PyTorch

3) Auto Classes的扩展与实现框架 15:31



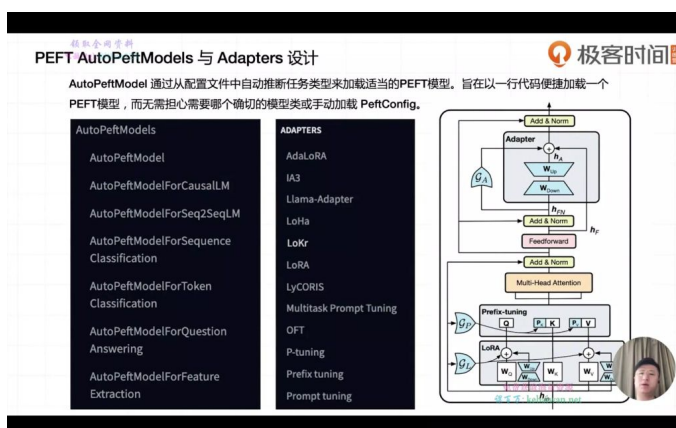
- 框架扩展机制：
 - 无前缀：PyTorch实现（原始默认）
 - TF前缀：TensorFlow实现（后期对接）
 - Flax前缀：JAX实现（含注意力机制优化）
- 配套组件：
 - AutoConfig：自动加载模型配置
 - AutoTokenizer：与模型严格匹配的文本处理器
 - AutoFeatureExtractor/AutoImageProcessor：跨模态特征处理器

4) Auto Classes的延展性 17:33

- 生态整合：
 - 通过from_pretrained()与HuggingFace Hub深度集成
 - 支持PEFT（参数高效微调）等扩展库的无缝接入
- 设计优势：
 - 统一接口：屏蔽底层框架差异（PyTorch/TF/JAX）
 - 模块化解耦：模型架构、任务头、预处理组件可独立替换
 - 自动发现机制：通过命名规则自动匹配预训练资源

5. PEFT AutoPeftModels与Adapters设计 17:56

1) PEFT高效微调方法与adapters 18:00



- 核心概念：PEFT的核心是各种高效微调方法，这些方法统称为adapters（适配器），包括IA3、LoRA、LoHa、LoKr、P-tuning、Prefix tuning、Prompt tuning、AdaLoRA等。
- 增量模型本质：所有PEFT方法都是在原始大模型基础上添加小型增量模块，如transformer结构中添加prefix/prompt或旁路添加LoRA矩阵。
- 适配器分类：主要分为三类：1) 前缀添加型（Prefix/Prompt tuning）2) 参数旁路型（LoRA）3) 内部插入型（Adapter tuning）。

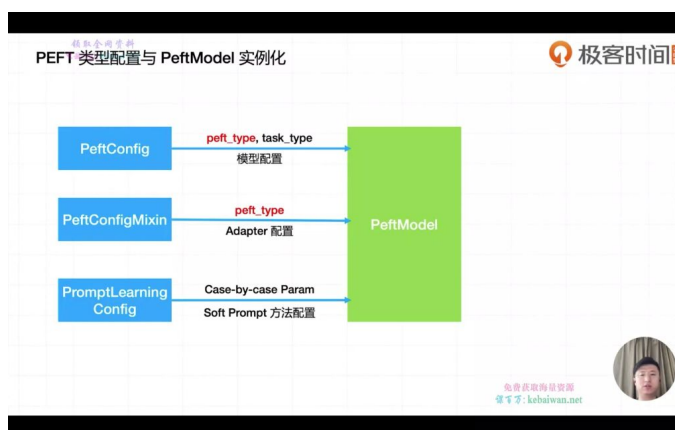
2) PEFT方法在AutoModel上的应用 18:28

- **实现方式**：通过transformers库的AutoModel中间插入"AutoPeft"实现，既不是后缀（如ForCausalLM）也不是前缀（如TFAutoModel）。
- **典型子类**：包括AutoPeftModelForCausalLM（因果语言模型）、AutoPeftModelForSeq2SeqLM（序列到序列）、AutoPeftModelForSequenceClassification（序列分类）等6大任务类型。

3) AutoPEFTModel的实现原理 20:18

- **包装机制**：如同"巧克力包糖纸"，基础模型（巧克力）通过PEFT配置（糖纸）包装成AutoPeftModel。
- **关键组件**：需要两个输入：1) 基础模型（来自transformers）2) PeftConfig（指定adapter类型和参数）。
- **参数特点**：新增参数量极少（通常仅原模型的0.1%-1%），如LoRA仅需训练低秩分解矩阵。

4) PEFT库的配置与AutoPEFTModel的包装 21:25

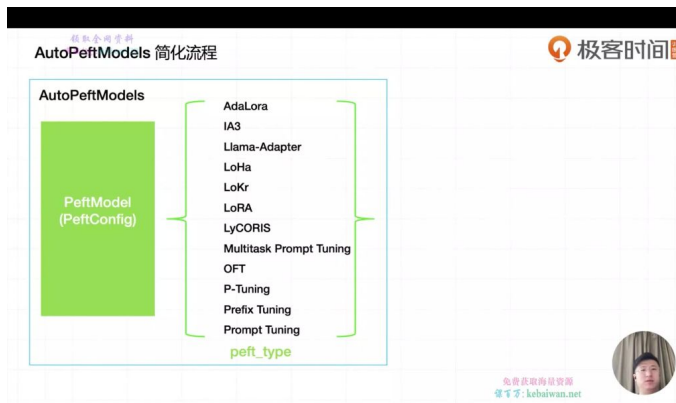


- **配置类型**：根据方法不同使用不同配置类：
 - 通用配置：PeftConfig（含peft_type和task_type）
 - Adapter专用：AdapterConfig（如LoRAConfig）
 - Prompt类：PromptLearningConfig（用于P-tuning等）
- **任务类型**：task_type对应下游任务（如CAUSAL_LM），可通过AutoPeftModelForXXX子类自动推断。

6. PEFT类型配置与PeftModel实例化 22:01

- **三要素配置**：
 - peft_type：指定微调方法（LoRA/IA3等）
 - task_type：明确下游任务（可省略）
 - 方法特定参数：如LoRA的秩r、alpha比例因子等
- **实例化流程**：
 - 创建基础模型（原始AutoModel）
 - 配置PeftConfig（含方法参数）
 - 调用get_peft_model()进行包装

7. 以task_type为核心简化PEFT流程 24:30



- 设计理念：延续transformers的AutoModelForXXX范式，通过task_type子类自动处理：
 - AutoPftModelForCausalLM
 - AutoPftModelForSeq2SeqLM
 - AutoPftModelForTokenClassification等
- 优势：与transformers生态完全兼容，可直接使用Trainer进行训练。

8. PEFT库微调工作原理与最佳实践 26:22

1) LoRA示例 26:23

PEFT 库微调工作原理与最佳实践（以 LoRA 为例）

极客时间

LoRA Adapter 配置

在 peft 中使用 LoRA 非常便捷，借助 PftModel 抽象，我们可以快速使用低秩适配器（LoRA）到任意模型。

通过使用 peft 中的 get_peft_model 工具函数来实现。

```

[8]: # 将peft库引入LoraConfig和get_peft_model函数
from peft import LoraConfig, get_peft_model

# 创建一个LoraConfig对象，用于设置LoRA (Low-Rank Adaptation) 的配置参数
config = LoraConfig(
    r=8, # LoRA的秩，影响LoRA矩阵的大小
    lora_alpha=32, # LoRA缩放因子
    # 指定在LoRA应用中使用的模型层，通常是指attention和全连接层的投影
    target_modules=["q_proj", "k_proj", "v_proj", "out_proj", "fc_in", "fc_out"],
    lora_dropout=0.05, # 在LoRA应用中使用的dropout率
    bias="none", # 设置bias的使用方式，这里没有使用bias
    task_type="CAUSAL_LM" # 任务类型，这里设置为因果语言模型
)

# 使用get_peft_model函数和给定的配置来创建一个PEFT模型
model = get_peft_model(model, config)

# 打印出模型中可训练的参数
model.print_trainable_parameters()
trainable params: 8,388,688 || all params: 6,666,862,592 || trainable: 0.12582542214183976 || 极客时间

```

极客时间

免费获取海量资源
微信号: kehuaiwan.net

- 关键参数：
 - $r = 8$ ：LoRA矩阵的秩（低维空间维度）
 - $lora_alpha = 32$ ：缩放因子（影响适应强度）
 - target_modules：指定应用模块（如["q_proj", "k_proj"]）
 - $lora_dropout = 0.05$ ：防止过拟合
- 代码示例：

```

1 from peft import LoraConfig, get_peft_model
2 config = LoraConfig(
3     r=8, lora_alpha=32,
4     target_modules=["q_proj", "k_proj"],
5     task_type="CAUSAL_LM"
6 )
7 peft_model = get_peft_model(base_model, config)
8 peft_model.print_trainable_parameters() # 示例输出：可训练参数800万/总参数60亿

```

- 参数占比：典型配置下可训练参数仅占总参数的0.125%（示例中8M/6.6B），实现高效微调。

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
PFT概念	Parameter Efficient Fine-Tuning (高效微调), 核心是不全量调整模型参数	区分PFT与传统微调 : PFT仅微调少量参数 (如1‰), 传统微调需调整全部参数	★★
PEFT库	Hugging Face开源的高效微调工具库, 集成多种SOTA方法 (如LoRA、Adapter)	支持模型与方法 : 通过官网工具可实时查询模型与PFT方法对应关系	★★
LoRA原理	通过低秩矩阵分解 ($B \times A$) 减少训练参数量, 核心参数: rank (降维维度)、alpha (比例因子)	参数设置 : target_modules选择需适配的权重矩阵 (如Q/K/V/FFN)	★★★
AutoModel分类	Transformers库中模型按结构分为: AutoModelForCausalLM (GPT类)、 AutoModelForMaskedLM (BERT类)、 AutoModelForSeq2SeqLM (T5类)	Head替换逻辑 : 下游任务决定模型头部结构 (如分类/生成)	★★★★
PEFT与Transformers集成	通过AutoPeftModel封装基础模型+Adapter (如LoRA), 使用get_peft_model()生成可训练模型	关键方法 : print_trainable_parameters()统计实际训练参数量 (如60亿→800万)	★★★
LoRA实战配置	LoraConfig需定义: r=8 (秩)、 target_modules=["q","v"]、 task_type="SEQ_CLS"	易错点 : bias参数通常不微调, dropout需根据任务调整	★★★★
任务类型适配	PFT支持任务分类与Transformers一致 (文本生成/分类/问答等), 通过task_type指定	跨模态支持 : LoRA通用性强, 可适配视觉 (ViT)、多模态模型	★★